Systém pro správu skladu elektronických součástek

WMS – Elektronické součástky

Projektová dokumentace

Autoři: Tomáš Simandl
Tomáš Szetei
Lukáš Kvapil

Datum: 5. dubna 2026

Dokumentace systému pro skladovou evidenci elektronických součástek

Obsah

1 Datový model **2**

1.1 Formulace problému 2

1.2 Datový slovník 2

1.3 Diagram tříd 4

1.4 Stavový diagram 6

1.5 Activity diagram – Vytvoření pohybu 8

1.6 Sekvenční diagram – Vytvoření pohybu 9

1.7 Use Case 10

Kapitola 1

Úvod

1.1 Motivace

Správa skladu elektronických součástek představuje náročný logistický úkol. Moderní elektronika využívá tisíce různých komponent – od běžných rezistorů a kondenzátorů přes specializované integrované obvody až po konektory a mechanické díly. Bez kvalitního informačního systému dochází k:

- duplicitním nákupům téhož zboží,
- ztrátě přehledu o aktuálních zásobách,
- zbytečnému zdržování při hledání součástek v neoznačených zásobnících,
- potížím při plánování výroby a prototypování,
- nesnadné dohledatelnosti dodavatelů a nákupních cen.

Systém WMS – Elektronické součástky si klade za cíl tyto problémy eliminovat a přinést přehlednou, efektivní a rozšiřitelnou správu skladových zásob.

1.2 Cíle projektu

1. Vytvořit centrální evidenci všech elektronických součástek včetně jejich technických parametrů.
2. Zavést přesné sledování fyzických umístění v skladovém prostoru (regály, přihrádky, zásobníky).
3. Automatizovat procesy příjmu a výdeje zboží s vazbou na projekty.
4. Umožnit rychlé vyhledávání součástek dle parametrů (hodnota, pouzdro, výrobce, kategorie).

5. Generovat reporty o stavu skladu a pohybech zásob.
6. Spravovat dodavatele a objednávky.
7. Upozorňovat na součástky pod minimální hranicí zásob.

1.3 Rozsah dokumentu

Tento dokument pokrývá:

- analýzu funkčních a nefunkčních požadavků (**??**),
- popis navrhované architektury systému (**??**),
- datový model a ER diagram (chapter 1),
- popis klíčových procesů formou UML diagramů (**??**).

1.4 Použité nástroje a technologie

Tabulka 1.1: Přehled použitých nástrojů

Oblast	**Nástroj / Technologie**	**Popis**
Dokumentace	LaTeX, Biber	Sazba odborného textu
Diagramy	PlantUML	UML a ER diagramy
Databáze	PostgreSQL / SQLite	Relační databáze
Backend	Python / FastAPI	REST API server
Frontend	React + TypeScript	Webová aplikace
Kontejnerizace	Docker, Docker Compose	Nasazení prostředí
Správa kódu	Git, GitHub	Verzování zdrojového kódu

Kapitola 2

Analýza požadavků

2.1 Funkční požadavky

2.1.1 Správa součástek

FR-01 Systém umožní evidovat součástku s atributy: název, typ, výrobce, hodnota, tolerance, pouzdro (package), popis, datasheet URL.

FR-02 Systém podporuje kategorie součástek (rezistory, kondenzátory, tranzistory, diody, IC, konektory, ...).

FR-03 Každá součástka může mít přiřazený čárový kód (EAN/QR).

FR-04 Uživatel může vyhledávat součástky dle libovolné kombinace parametrů.

2.1.2 Skladová místa

FR-05 Systém spravuje hierarchii skladových míst: *Sklad* → *Sekce* → *Regál* → *Přihrádka*.

FR-06 Jedno skladové místo může obsahovat více druhů součástek; jedna součástka může být na více místech.

2.1.3 Příjem a výdej

FR-07 Příjem součástek ze skladu generuje pohyb typu `PŘÍJEM` a aktualizuje zásoby.

FR-08 Výdej součástek na projekt generuje pohyb typu `VÝDEJ` a sníží zásoby.

FR-09 Přeskladnění přesouvá zásoby mezi místy bez změny celkového množství.

FR-10 Systém hlídá minimální zásoby a upozorní na podkročení limitu.

2.1.4 Objednávky a dodavatelé

FR-11 Systém eviduje dodavatele s kontaktními údaji.

FR-12 Uživatel může vytvořit nákupní objednávku u dodavatele.

FR-13 Objednávka prochází stavy: `NÁVRH` → `ODESLÁNO` → `POTVRZENO` → `PŘIJATO` → `STORNOVÁNO`.

2.1.5 Inventura

FR-14 Systém umožní zahájit inventurní cyklus, během nějž uživatel ověřuje fyzické zásoby.

FR-15 Systém generuje inventurní list (PDF/CSV) se seznamem součástek a očekávaných množství.

FR-16 Po ukončení inventury systém provede korekci zásob.

2.1.6 Reporty

FR-17 Přehled aktuálních zásob s filtrací dle kategorie/umístění.

FR-18 Historie pohybů za zvolené časové období.

FR-19 Součástky pod minimálním limitem zásob.

FR-20 Ocenění skladu (celková hodnota zásob).

2.1.7 Správa uživatelů

FR-21 Systém spravuje uživatele s rolemi: `ADMIN`, `SKLADNÍK`, `ČTENÁŘ`.

FR-22 Každá akce je logována s časovým razítkem a uživatelem.

2.2 Nefunkční požadavky

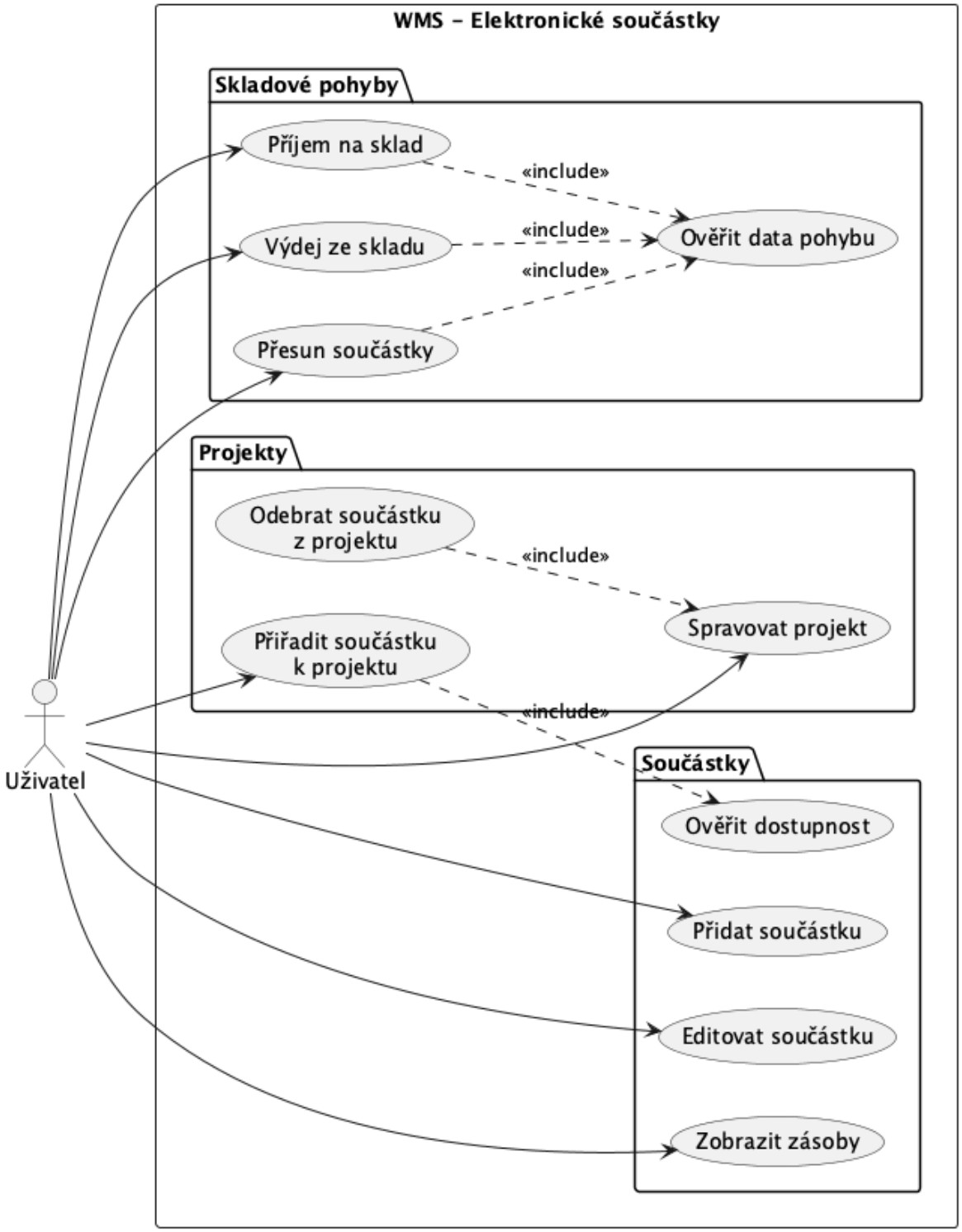
Tabulka 2.1: Nefunkční požadavky systému

ID	Kategorie	Požadavek
NFR-01	Výkon	Vyhledávání vrátí výsledky do 500 ms pro zásoby do 100 000 položek.
NFR-02	Dostupnost	Systém musí být dostupný 99,5 % času (plánovaná údržba vyjmuta).
NFR-03	Bezpečnost	Hesla uložena jako bcrypt hash; komunikace pouze přes HTTPS.
NFR-04	Škálovatelnost	Architektura umožní horizontální škálování backendu.
NFR-05	Přenositelnost	Nasazení pomocí Docker Compose; nezávislé na OS.
NFR-06	Použitelnost	Webové rozhraní přístupné bez speciálního SW v moderním prohlížeči.
NFR-07	Lokalizace	Primární jazyk čeština; systém připraven na vícejazyčnost (i18n).
NFR-08	Auditovatelnost	Každá změna stavu skladu musí být dohledatelná v auditním logu.

2.3 Případy užití – přehled

Hlavní aktér systému je **Uživatel**, který odpovídá třídě `Uzivatel` v datovém modelu. Uživatel provádí veškeré operace se systémem – vytváří skladové pohyby, spravuje součástky i projekty.

Podrobný Use Case diagram je zobrazen v **??**.



Obrázek 2.1: Use Case diagram systému WMS – elektronické součástky

Kapitola 3

Datový model

3.1 Formulace problému

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi.

3.2 Datový slovník

Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Atribut `rezervovaneMnozstvi` sleduje, kolik kusů je aktuálně blokováno pro projekty – součástka tak může být *Dostupna* i s částečnou rezervací zároveň. Součástka může nabývat stavů: *Dostupna* (k dispozici, případně s rezervovanou částí), *Vydana* (fyzicky vydána ze skladu) nebo *Odepsana* (trvale vyřazena z evidence).

- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.

- **SkladovyPohyb**: Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun.

- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele (dědí ze **SkladovyPohyb**).

- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu (dědí ze **SkladovyPohyb**). Atribut `duvod` popisuje účel výdeje, např. *použití v projektu*, *reklamace*, *interní spotřeba* nebo *oprava*.
- **PresunutiPohyb**: Konkrétní pohyb typu přesun – přemístnění součástek mezi lokáními ve skladu (dědí ze **SkladovyPohyb**).
- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

Atributy

Tabulka 3.1: Přehled atributů tříd

Entita	**Atributy**
Soucastka	id, nazev, mnozstvi, rezervovaneMnozstvi, stav (Dostupna | Vydana | Odepsana)
Projekt	id, nazev
SkladovyPohyb	id, datum, stav, typ (prijem | vydej | presun), mnozstvi
PrijemPohyb	dodavatel – vlastní; id, datum, stav, typ – zděděno
VydejPohyb	duvod – vlastní; id, datum, stav, typ – zděděno
PresunutiPohyb	zdrojLokace, cilLokace – vlastní; id, datum, stav, typ – zděděno
Sklad	id, nazev
Lokace	id, oznaceni
Uzivatel	id, jmeno, email

3.3 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelské roli v systému. Vytvoření pohybu provádí voláním `SkladovyPohyb.vytvoritPohyb()` a přiřazení součástky k projektu voláním `Projekt.priradSouca`

Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída `SkladovyPohyb` deklaruje abstraktní metodu `vytvoritPohyb()`, kterou každá podtřída implementuje po svém:

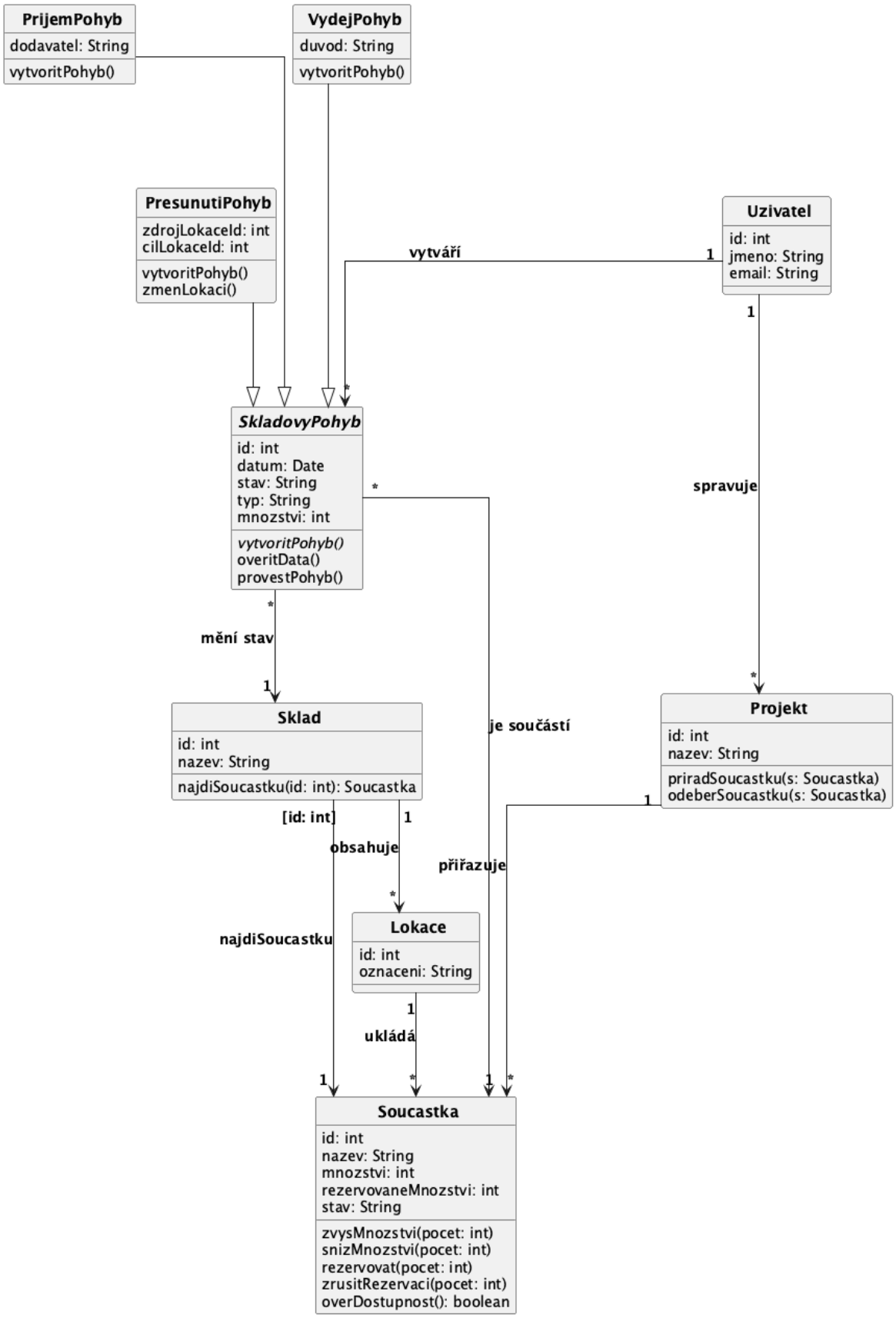
- `PrijemPohyb.vytvoritPohyb()` – vytvoří příjem a nastaví atribut `dodavatel`.
- `VydejPohyb.vytvoritPohyb()` – vytvoří výdej a nastaví atribut `duvod`.
- `PresunutiPohyb.vytvoritPohyb()` – vytvoří přesun a nastaví `zdrojLokaceId` a `cilLokaceId`.

Třída `SkladovyPohyb` zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- `PrijemPohyb` – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- `VydejPohyb` – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- `PresunutiPohyb` – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba `je součástí`, multiplicita $* \rightarrow 1$) a mění stav jednoho skladu (vazba `mění stav`, multiplicita $* \rightarrow 1$). Jeden uživatel může vytvořit libovolný počet pohybů (vazba `vytváří`, multiplicita $1 \rightarrow *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou `Lokace` $\rightarrow$ `Soucastka`.

Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.



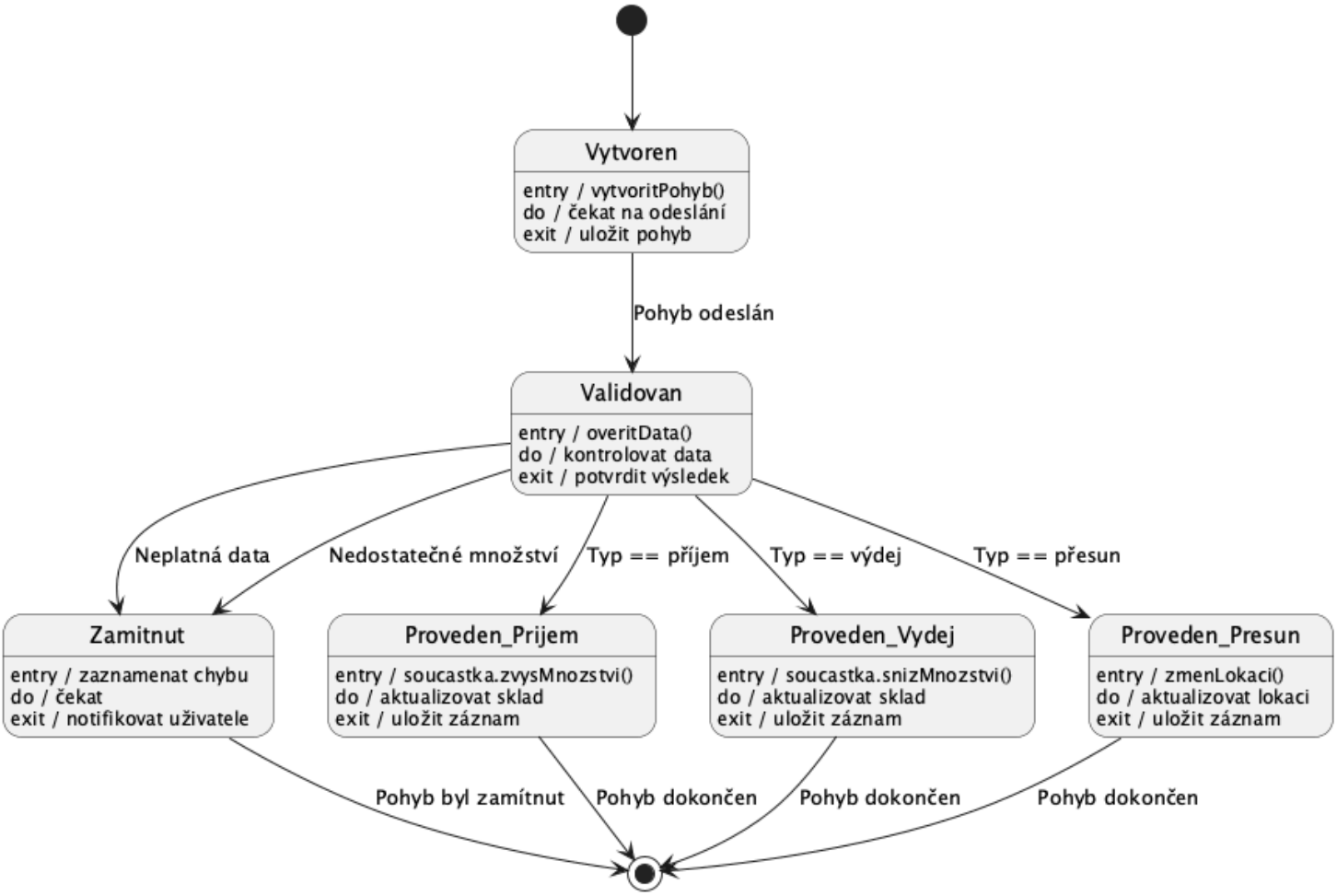
Obrázek 3.1: Class diagram WMS

3.4 Stavový diagram

Stavový diagram třídy `SkladovyPohyb`

Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- `vytvoritPohyb()` – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- `overitData()` – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- `provestPohyb()` – fyzicky provede pohyb: u příjmu zavolá `soucastka.zvysMnozstvi()`, u výdeje `soucastka.snizMnozstvi()` a u přesunu `zmenLokaci()`.

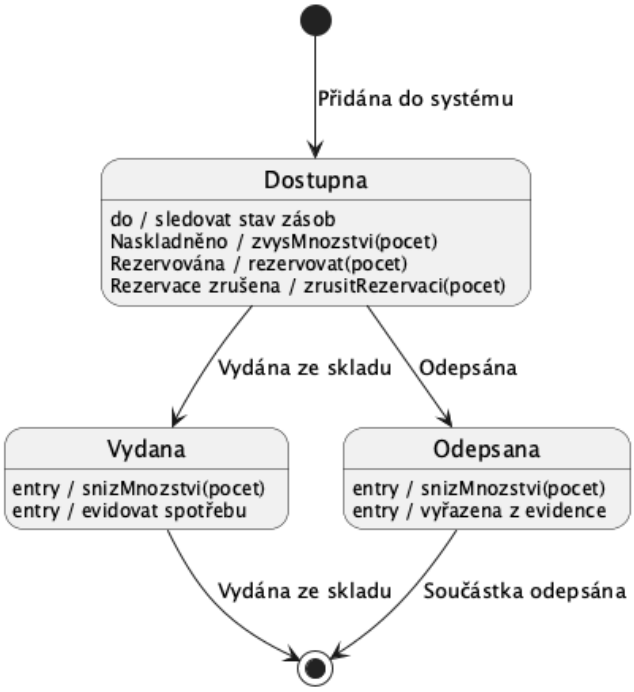


Obrázek 3.2: Stavový diagram třídy SkladovyPohyb

Stavový diagram třídy `Soucastka`

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je přidána do systému a okamžitě vstupuje do stavu *Dostupna* s nulovým množstvím. Stav *Dostupna* je hlavní pracovní stav součástky – může mít `mnozstvi = 0` (čeká na naskladnění) i nenulové množství, přičemž část může být blokována atributem `rezervovaneMnozstvi`. Metoda `overDostupnost()` ověří, zda je volné množství (`mnozstvi - rezervovaneMnozstvi > 0`). Stav součástky je řízen těmito metodami:

- `zvysMnozstvi(pocet)` – zavolá se při naskladnění; zvýší atribut `mnozstvi`, součástka zůstává ve stavu *Dostupna*.
- `rezervovat(pocet)` – zablokuje část kusů pro projekt; zvýší `rezervovaneMnozstvi`, stav se nemění.
- `zrusitRezervaci(pocet)` – uvolní rezervaci; sníží `rezervovaneMnozstvi`, stav se nemění.
- `snizMnozstvi(pocet)` – zavolá se při výdeji ze skladu (*Vydana*) nebo odpisu (*Odepsana*); oba jsou terminální stavy.

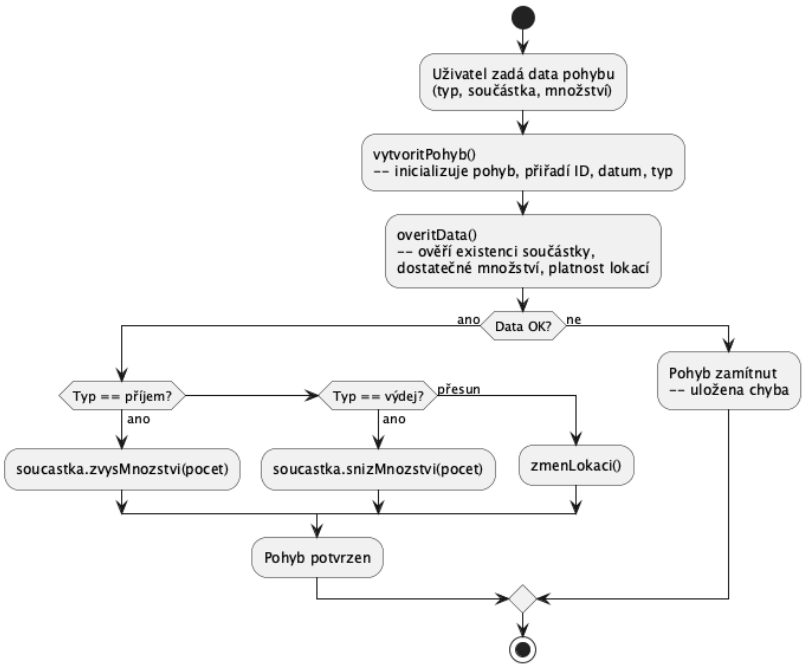


Obrázek 3.3: Stavový diagram třídy Soucastka

3.5 Activity diagram – Vytvoření pohybu

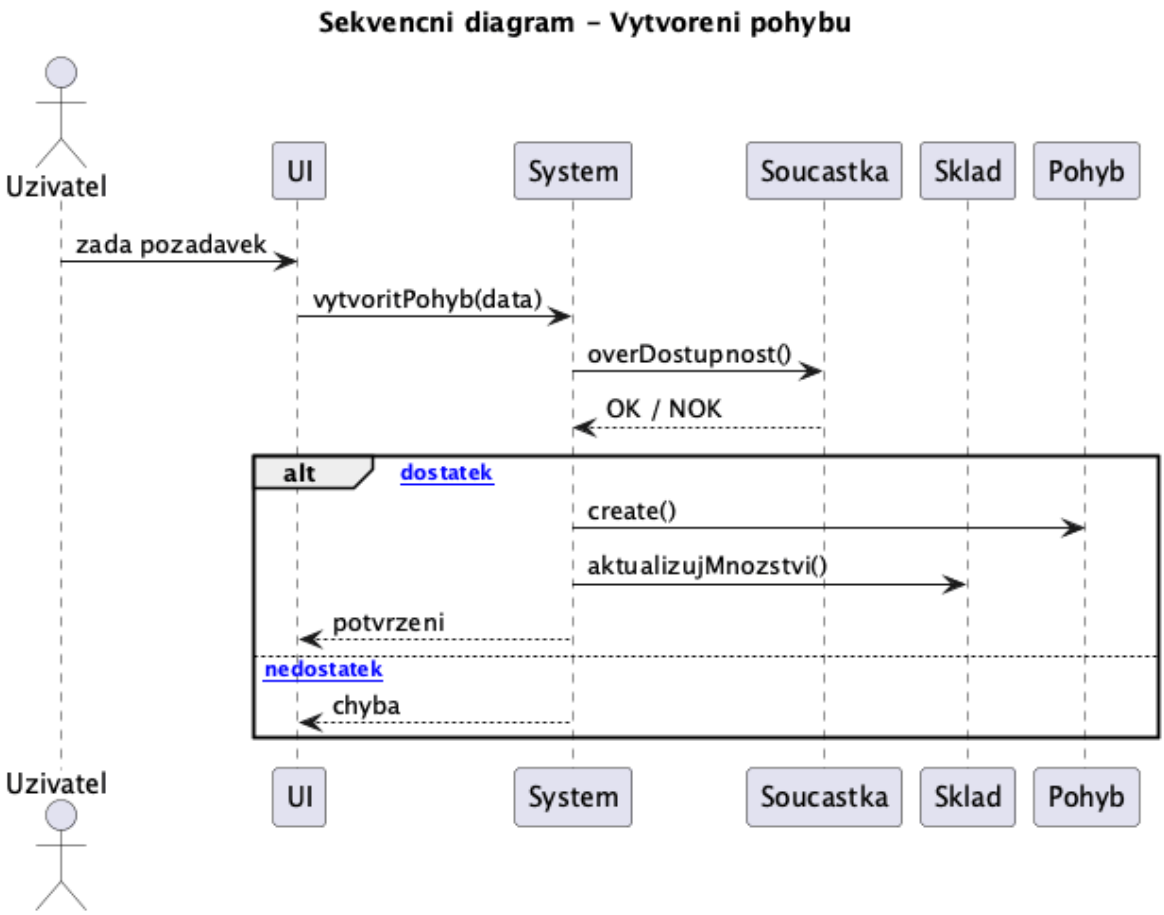
Diagram zachycuje průběh vytvoření skladového pohybu v systému WMS. Pohyb prochází třemi fázemi řízenými metodami třídy `SkladovyPohyb`:

- `vytvoritPohyb()` – uživatel zadá typ pohybu, součástku a množství; metoda inicializuje pohyb, přiřadí ID a datum vzniku.
- `overitData()` – ověří platnost zadaných dat: existenci součástky, dostatečné množství na skladě a platnost lokací u přesunu.
- `provestPohyb()` – fyzicky provede pohyb podle typu: příjem zavolá `zvysMnozstvi(pocet)`, výdej `snizMnozstvi(pocet)`, přesun `zmenLokaci()`. Pokud data nejsou platná, pohyb je zamítnut a uložena chyba.



Obrázek 3.4: Activity diagram – Vytvoření pohybu

3.6 Sekvenční diagram – Vytvoření pohybu



Obrázek 3.5: Sekvenční diagram – Vytvoření pohybu

3.7 Use Case

Use Case diagram systému je zobrazen v **??** (kapitola Analýza požadavků). Z pohledu datového modelu jsou klíčové tyto scénáře, které přímo odpovídají metodám tříd `SkladovyPohyb` a `Projekt`:

Scénář: Vytvoření pohybu

1. Uživatel zvolí typ pohybu – příjem (`PrijemPohyb`), výdej (`VydejPohyb`) nebo přesun (`PresunutiPohyb`).
2. Zadá součástku a množství.
3. Systém volá `overitData()` – ověří existenci součástky, dostatečné množství a platnost lokací.
4. Pokud data OK → systém volá `provestPohyb()` a aktualizuje zásoby.
5. Pohyb je potvrzen; v případě chyby je zamítnut.

Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt a zadá součástku s množstvím.
2. Systém volá `overDostupnost()` – ověří, zda `mnozstvi - rezervovaneMnozstvi > 0`.
3. Pokud je dostatek volných kusů → systém volá `Projekt.priradSoucastku()`.
4. Součástka je přiřazena k projektu; zásoby jsou aktualizovány.

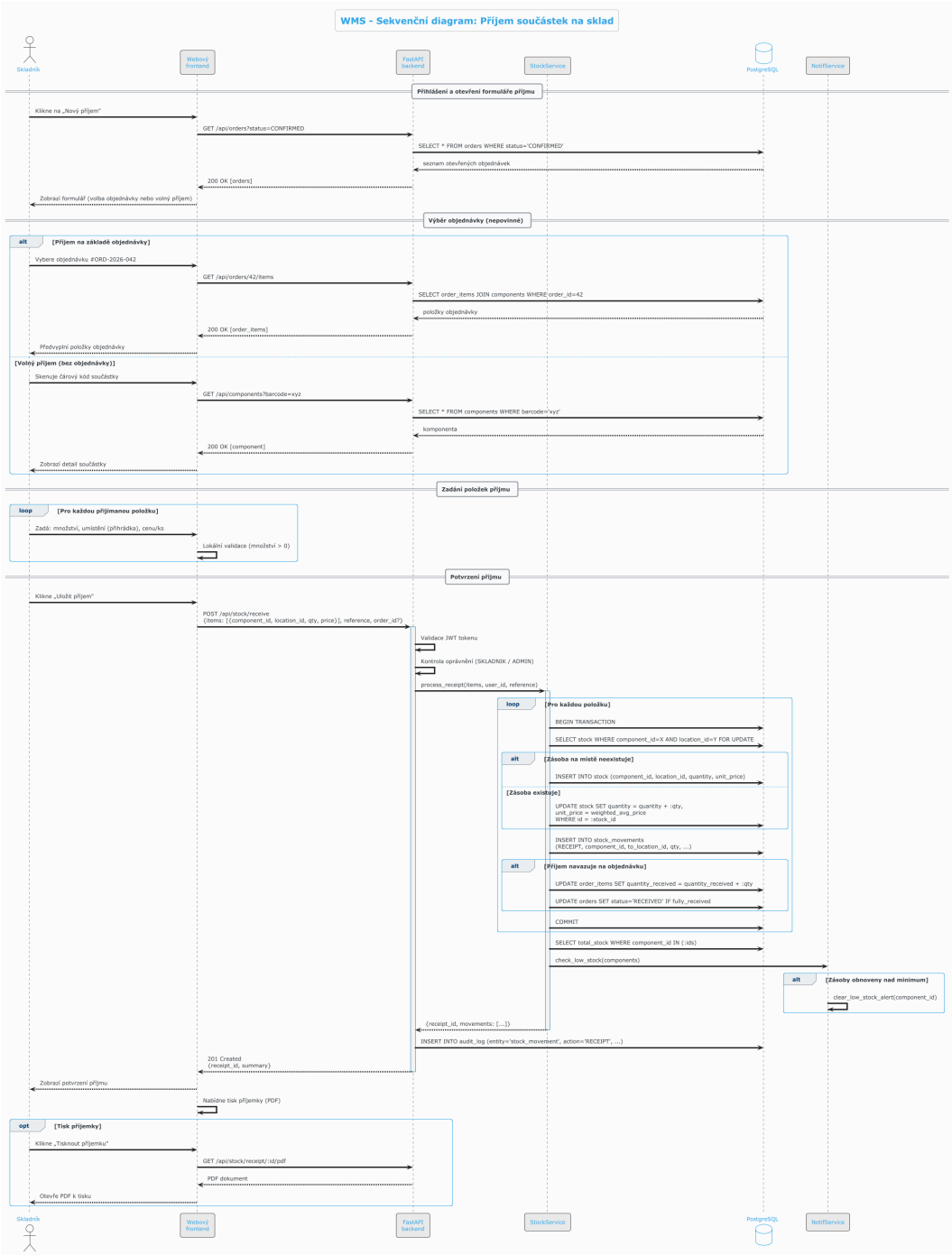
Kapitola 4

Popis procesů

4.1 Proces příjmu součástek

Příjem součástek na sklad je jedním z nejčastějších procesů. Probíhá typicky na základě dodávky od dodavatele, přičemž může, ale nemusí, navazovat na předchozí objednávku.

Sekvenční diagram příjmu je zobrazen na **??**.



Obrázek 4.1: Sekvenční diagram – Příjem součástek na sklad

4.1.1 Popis kroků příjmu

1. Skladník otevře formulář příjmu a naskenuje/zadá číslo dokladu nebo vyhledá objednávku.

2. Pro každou přijímanou položku:

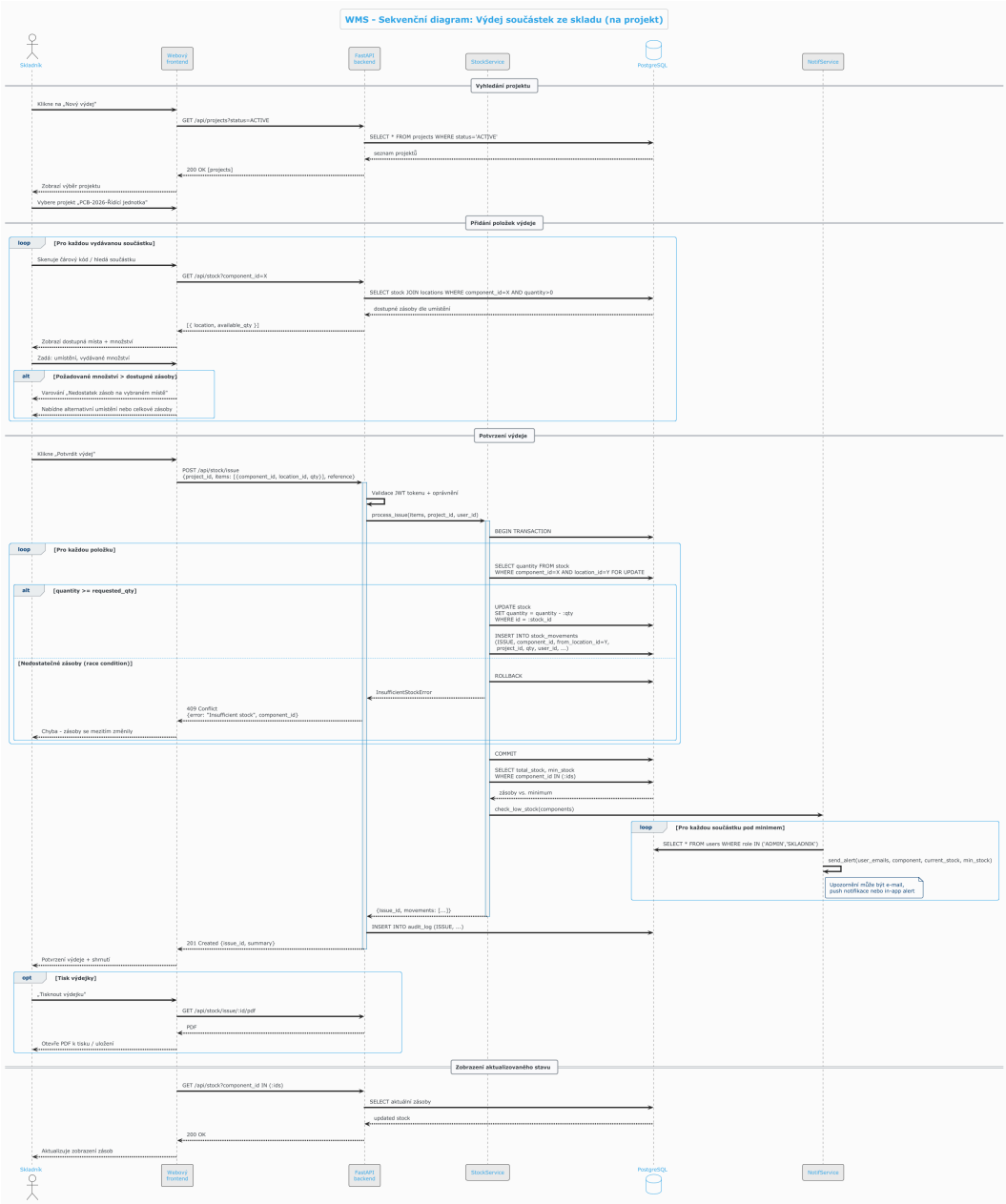
   (a) Vyhledá součástku (dle part number nebo skenování čárového kódu).

   (b) Určí cílové skladové místo.
   (c) Zadá přijímané množství a jednotkovou cenu.

3. Systém ověří validitu dat (množství > 0, místo existuje, součástka aktivní).
4. Systém vytvoří záznamy v `stock_movements` a aktualizuje tabulku `stock`.
5. Pokud příjem navazuje na objednávku, systém aktualizuje stav objednávky.
6. Systém zkontroluje, zda příjem neobnovil zásoby nad minimum (zruší upozornění).
7. Vytiskne se příjemka.

4.2 Proces výdeje součástek

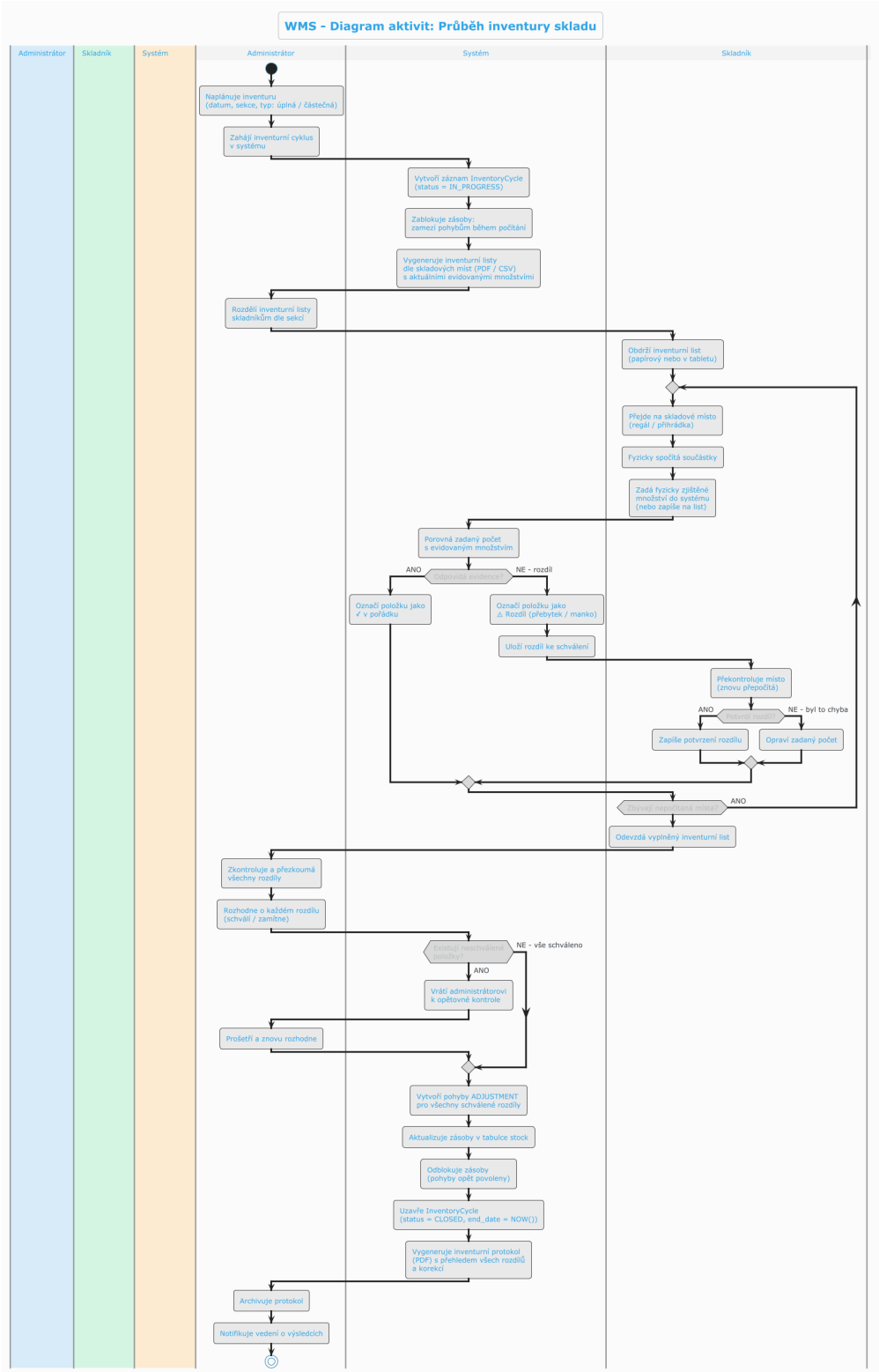
Výdej se provádí při přidělení součástek konkrétnímu projektu nebo opravě. Sekvenční diagram je na **??**.



Obrázek 4.2: Sekvenční diagram – Výdej součástek ze skladu

4.3 Proces inventury

Inventura je periodický proces ověření fyzického stavu skladu. Diagram aktivit je zobrazen na ??.



Obrázek 4.3: Diagram aktivit – Průběh inventury

4.3.1 Fáze inventurního procesu

Příprava Administrátor zahájí inventurní cyklus. Systém uzamkne zásoby pro editaci a vygeneruje inventurní listy dle skladových míst.

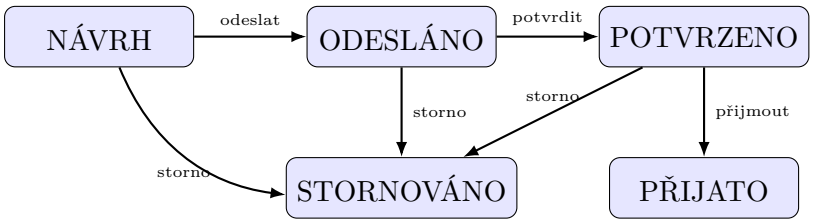
Fyzické sčítání Skladník prochází místa a zadává fyzicky zjištěné množství. Systém umožňuje zadávání postupně po sekcích.

Porovnání Systém porovná evidované a fyzicky zjištěné stavy. Zobrazí seznam rozdílů (přebytky / manka).

Schválení a korekce Administrátor přezkoumá rozdíly. Po schválení systém vytvoří pohyby typu `ADJUSTMENT` a aktualizuje zásoby.

Uzavření Inventura je uzavřena, zásoby odemčeny, vygenerován protokol.

4.4 Stav objednávky – stavový diagram



Obrázek 4.4: Stavový diagram objednávky

Kapitola 5

Architektura systému

5.1 Přehled architektury

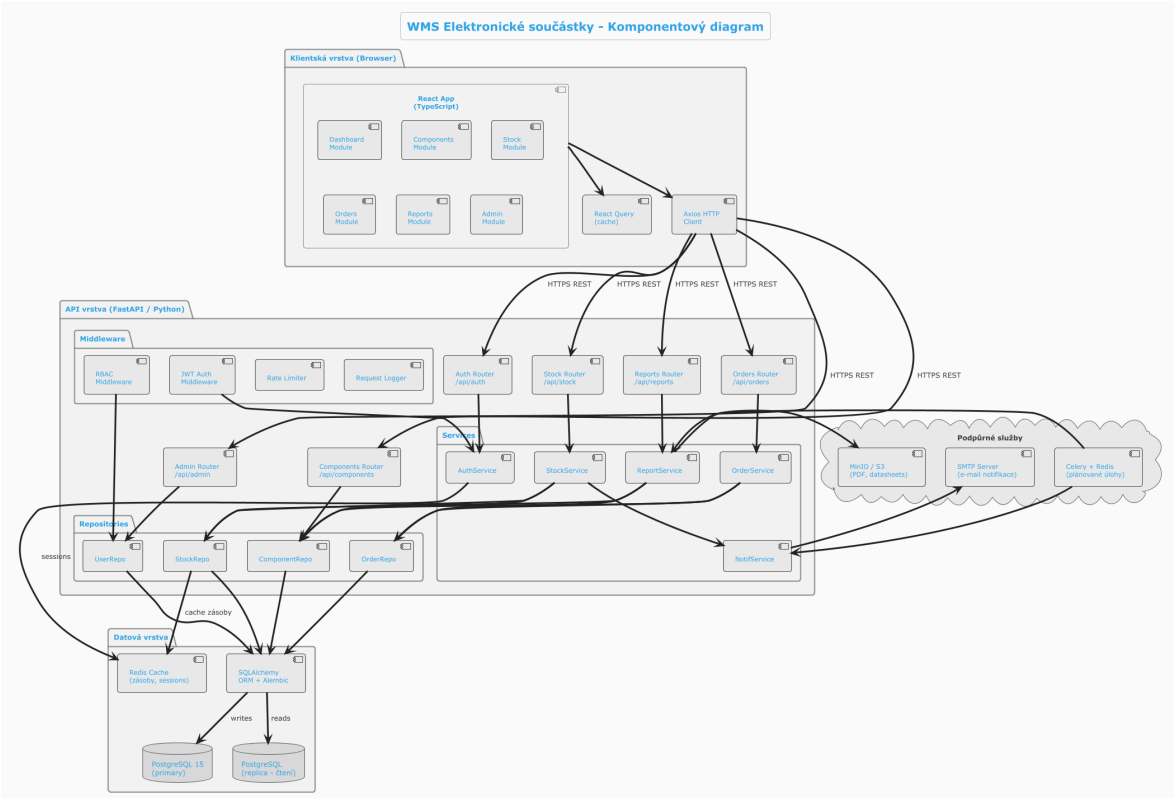
Systém je navržen jako třívrstvá webová aplikace s jasně oddělenými vrstvami:

Prezentační vrstva Webová aplikace napsaná v React + TypeScript, která komunikuje s backendem prostřednictvím REST API.

Aplikační vrstva Python backend s frameworkem FastAPI implementující business logiku, autentizaci (JWT) a validaci dat.

Datová vrstva Relační databáze správující persistenci dat; přístup přes ORM SQLAlchemy.

Komponentový diagram systému je zobrazen v **??**.



Obrázek 5.1: Komponentový diagram systému

5.2 REST API – přehled endpointů

Tabulka 5.1: Klíčové REST API endpointy

Metoda	Endpoint	Popis
GET	`/api/components`	Seznam součástek (s filtrací)
POST	`/api/components`	Nová součástka
GET	`/api/components/{id}`	Detail součástky
PUT	`/api/components/{id}`	Aktualizace součástky
DELETE	`/api/components/{id}`	Smazání součástky
GET	`/api/stock`	Aktuální zásoby
POST	`/api/stock/receive`	Příjem na sklad
POST	`/api/stock/issue`	Výdej ze skladu
POST	`/api/stock/transfer`	Přeskladnění
GET	`/api/locations`	Skladová místa
GET	`/api/suppliers`	Dodavatelé
GET	`/api/orders`	Objednávky
POST	`/api/orders`	Nová objednávka
PATCH	`/api/orders/{id}/status`	Změna stavu objednávky
GET	`/api/reports/stock`	Report stavu skladu
GET	`/api/reports/movements`	Report pohybů
GET	`/api/reports/low-stock`	Součástky pod limitem

5.3 Bezpečnostní model

Autentizace je realizována pomocí JWT tokenů (JSON Web Token):

1. Uživatel odešle přihlašovací údaje na `POST /api/auth/login`.
2. Server ověří heslo (bcrypt), vygeneruje access token (platnost 15 min) a refresh token (platnost 7 dní).
3. Klient přikládá access token v hlavičce `Authorization: Bearer {token}`.
4. Po vypršení access tokenu si klient vyžádá nový pomocí refresh tokenu.

Oprávnění jsou řešena pomocí RBAC (Role-Based Access Control):

Tabulka 5.2: Matice oprávnění dle rolí

Operace	**Čtenář**	**Skladník**	**Admin**
Prohlížení součástek	✓	✓	✓
Editace součástek	—	✓	✓
Příjem / Výdej	—	✓	✓
Inventura	—	✓	✓
Správa dodavatelů	—	✓	✓
Správa uživatelů	—	—	✓
Konfigurace systému	—	—	✓

Kapitola 6

Formální specifikace dotazů

Tato kapitola zavádí formální notaci pro popis dotazů nad datovým modelem WMS. Notace je inspirována lambda kalkulem a množinovou algebrou a je platformově nezávislá.

6.1 Definice notace

Universum. $\mathcal{U}$ označuje množinu všech uzlů (objektů) v systému. Množiny jsou reprezentovány jako charakteristické funkce; $x \in A$ je zkrácený zápis za $(A\ x)$.

Selekce (filtr). Infixový operátor // filtruje množinu:

$$A \mathbin{//} \Lambda \;\coloneqq\; \{\, x \mid (x \in A) \wedge (\Lambda\, x) \,\}$$

Transformace (map). Infixový operátor $\gg$ mapuje množinu:

$$A \gg f \;\coloneqq\; \{\, f(x) \mid x \in A \,\}$$

Flatten. Funkce flatten sloučí množinu množin:

$$\text{flatten}(M) \;=\; \bigcup M$$

Skládání do struktury. Infixový operátor $\oplus$ vytváří strukturu z hodnot:

$$a \oplus b \;\coloneqq\; \text{Struct}(a,\, b)$$

Atributy. Tečková notace je zkratkou pro funkční zápis: $s.věk \coloneqq věk(s)$.

Orientovaná hrana. $x \xrightarrow{\text{Relace}} y$ znamená $(x, y) \in \text{Relace}$.

6.2 Příklady dotazů nad modelem WMS

6.2.1 Součástky pod minimálním stavem

Množina součástek, jejichž celkový stav zásob klesl pod minimum:

$$K_{\min} \;=\; \text{Komponenty} \,//\, \Big(\lambda k \mid k.totalStock < k.minStock\Big)$$

kde $k.totalStock$ je agregovaná hodnota:

$$k.totalStock \;:=\; \sum_{z\,:\,k \xrightarrow{\text{ULOZENA_NA}} z} z.quantity$$

Výstupní projekce (jméno, číslo, stav):

$$K_{\min} \gg \Big(\lambda k \mid k.name \oplus k.partNumber \oplus k.totalStock\Big)$$

6.2.2 Celková hodnota skladu

Suma hodnot všech zásob:

$$HodnotaSkladu \;=\; \sum_{z \in \text{Zásoby}} z.quantity \times z.unitPrice$$

6.2.3 Katalog součástek dodavatele

Všechny součástky nabízené konkrétním dodavatelem d_0:

$$\text{flatten}\Big(\Big\{\, d_0 \,\Big\} \gg \Big(\lambda d \mid \Big\{\, k \mid d \xrightarrow{\text{DODAVANA_OD}} k \Big\}\Big)\Big)$$

6.2.4 Pohyby výdeje na projekt

Pohyby typu `VYDEJ` navázané na projekt p_0, provedené za dané časové období $[t_1, t_2]$:

$$\text{Pohyby}//\Big(\lambda m \mid m.movementType = \texttt{VYDEJ} \wedge m \xrightarrow{\text{VYDANA_NA}} p_0 \wedge t_1 \leq m.createdAt \leq t_2\Big)$$

Výsledná projekce (součástka, množství, datum):

$$\ldots \gg \Big(\lambda m \mid m.component \oplus m.quantity \oplus m.createdAt\Big)$$

6.2.5 Složité dotazy – inventurní rozdíly

Položky inventury, kde se zjištěný počet liší od evidovaného (schválené i neschválené):

$$\text{PolozkyInventury}//\Big(\lambda p \mid p.countedQty \neq p.expectedQty\Big) \gg \Big(\lambda p \mid p.component \oplus p.expectedQty \oplus p.$$

Kapitola 7

Závěr

7.1 Shrnutí

Tato dokumentace popsala návrh systému pro správu skladu elektronických součástek (WMS). Systém řeší reálnou potřebu přehledné a efektivní evidence zásob v prostředí, kde se pracuje s tisíci různých komponent.

Klíčové výstupy projektu:

- Analýza 22 funkčních a 8 nefunkčních požadavků.
- Navžena třívrstvá architektura (React + FastAPI + relační DB).
- Datový model s 11 hlavními entitami pokrývající kompletní lifecycle součástky v skladu.
- 7 UML/PlantUML diagramů dokumentujících use cases, konceptuální model, ER diagram, sekvenční a aktivitní procesy a komponentovou architekturu.
- REST API s více než 15 endpointy.

7.2 Další rozvoj

Plánovaná rozšíření pro budoucí verze systému:

v1.1 Mobilní aplikace pro skenování QR kódů při příjmu a výdeji.

v1.2 Integrace s online obchody (TME, Farnell, Mouser) pro automatické doplnění technických parametrů a cen.

v1.3 BOM (Bill of Materials) management – propojení WMS s návrhem DPS pro automatické ověření dostupnosti součástek.

v2.0 Multi-sklady a multi-uživatelský přístup s pokročilými právy. Notifikace přes e-mail a Teams/Slack.

7.3 Diagram tříd – rozšiřitelnost

Navržená architektura je připravena na budoucí rozšíření díky použití abstraktních bázových tříd a dependency injection v backendu, což umožní přidávat nové typy pohybů a reportů bez zásahu do existujících modulů.